

Experiment 10: Map Coloring using CSP

Aim

To implement the Map Coloring problem using Constraint Satisfaction Problem (CSP).

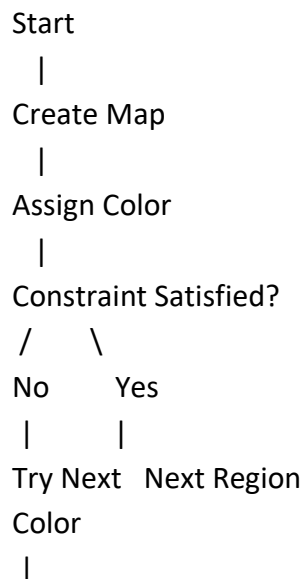
Objective

- To assign colors to regions.
- To ensure adjacent regions have different colors.

Algorithm

1. Create the map and adjacency list.
2. Define available colors.
3. Assign colors recursively.
4. Check whether neighboring regions have different colors.
5. If valid, continue.
6. Display the final coloring.

Flowchart



```
Repeat
|
Display Colors
|
End
```

Program

```
colors = ["Red","Green","Blue"]
```

```
states = ["A","B","C","D"]
```

```
neighbors = {
"A":["B","C"],
"B":["A","C","D"],
"C":["A","B","D"],
"D":["B","C"]
}
```

```
assignment = {}
```

```
def valid(state,color):
    for n in neighbors[state]:
        if assignment.get(n)==color:
            return False
    return True
```

```
def solve(i):
    if i==len(states):
        return True
```

```
    state=states[i]
```

```
    for color in colors:
        if valid(state,color):
            assignment[state]=color
            if solve(i+1):
                return True
```

```
        del assignment[state]  
    return False
```

```
solve(0)
```

```
print(assignment)
```

Output

```
{'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}
```

Result

The map was successfully colored without violating any constraints.

Conclusion

The CSP approach successfully assigns colors so that no two adjacent regions share the same color.